

Unidad 7

Fundamentos, Enfoques y Niveles de Pruebas

1. Introducción: La Mentalidad del Tester

1.1. ¿Por Qué Probar el Software?

En las primeras unidades de esta materia, hemos estudiado cómo planificar y construir software. Ahora, debemos abordar una pregunta fundamental: **¿Cómo sabemos que lo que construimos funciona bien?**

El **Testing de Software** (o Pruebas de Software) es el proceso de evaluar un sistema o sus componentes con la intención de encontrar si satisface los requisitos especificados.

Es un error común pensar que el objetivo del testing es "buscar errores" o "romper el software". La meta real es mucho más profunda y profesional:

- **Medir la Calidad:** Obtener información objetiva sobre qué tan bien (o mal) funciona el producto. Las pruebas son una herramienta de medición.
- **Reducir el Riesgo:** Identificar problemas *antes* de que el usuario final los encuentre. Probar el software protege al negocio de fallas catastróficas, pérdidas de datos, problemas legales o daño a su reputación.
- **Confirmar Requisitos:** Asegurar que el software que *construimos* es el software que el cliente *pidió*.

Probar el software no es una fase final; es una actividad de investigación continua que nos da la confianza para lanzar un producto.

1.2. El Costo Exponencial de un Defecto (Bug)

Una de las principales justificaciones para invertir tiempo en pruebas es económica. En la ingeniería de software, existe una regla de oro demostrada: **el costo de corregir un defecto (bug) aumenta exponencialmente a medida que el proyecto avanza.**

Pensemos en un requisito malentendido por el equipo:

- **Fase de Requisitos:** Corregirlo es solo una aclaración en una reunión.
 - **Costo: \$1**
- **Fase de Diseño:** Corregirlo implica rehacer diagramas de arquitectura y flujos.
 - **Costo: \$10**

- **Fase de Desarrollo:** Corregirlo implica borrar y reescribir módulos de código.
 - **Costo: \$100**
- **Fase de Pruebas (QA):** Corregirlo implica que el tester lo encuentre, lo reporte, el desarrollador lo corrija y el tester lo vuelva a probar (re-testing).
 - **Costo: \$1.000**
- **En Producción (Cliente):** El cliente descubre el error. Implica pérdida de datos, quejas de usuarios, daño a la reputación y una corrección "de emergencia" que interrumpe todo el trabajo planificado.
 - **Costo: \$10.000+**

El testing no es un gasto, es una inversión. Invertir \$100 en la Fase de Desarrollo (con pruebas) nos ahorra \$10.000 en la Fase de Producción.

1.3. La Psicología del Testing: La Curiosidad Disciplinada

Probar requiere una mentalidad específica, que a menudo es complementaria (y opuesta) a la del desarrollo.

- **La Mentalidad del Desarrollador:** Es *constructiva*. Su objetivo es crear algo que funcione. Tiende a probar el "camino feliz" (el usuario hace todo bien) para demostrar que su código cumple la tarea.
- **La Mentalidad del Tester:** Es *investigativa*. Su objetivo es descubrir escenarios donde el código *no funciona*. Es un escepticismo saludable, una "curiosidad disciplinada".

Un buen tester no dice "Voy a romper la app". Un buen tester pregunta: "**¿Qué pasa si...?**"

- "*¿Qué pasa si el usuario presiona 'Guardar' dos veces muy rápido?*"
- "*¿Qué pasa si el campo 'nombre' está vacío, pero el campo 'apellido' no?*"
- "*¿Qué pasa si el usuario ingresa emojis 🤪 o caracteres chinos (汉字) en el campo 'DNI'?*"
- "*¿Qué pasa si intento comprar un producto con stock negativo o con cantidad 0.5?*"
- "*¿Qué pasa si pierdo la conexión a internet justo antes de presionar 'Pagar'?*"

Para aplicar esta curiosidad de forma estructurada, la ingeniería de software define tres **Enfoques** principales.

2. Enfoques Fundamentales: ¿Desde Dónde Probamos?

Los enfoques responden a la pregunta: ¿cuánta información sobre el *interior* del sistema tiene el tester?

2.1. Pruebas de Caja Negra (Black-Box Testing)

Este es el enfoque más intuitivo. Las pruebas de Caja Negra tratan al software como una "caja" opaca. **No tenemos idea de lo que hay dentro.**

El tester no ve el código fuente, no sabe en qué lenguaje está programado, ni cómo está diseñada la base de datos.

El objetivo es probar la **funcionalidad** del sistema basándonos únicamente en sus **entradas** (inputs) y sus **salidas esperadas** (outputs).

- **Se enfoca en:** "¿El sistema *hace* lo que se supone que debe hacer?"
- **Perspectiva:** La del **Usuario Final**.
- **Se basa en:** Los requisitos del sistema.
- **Analogía:** Probar un cajero automático. Ingresas tu tarjeta (entrada), tu PIN (entrada) y pides \$20.000 (entrada). Esperas que la máquina entregue \$20.000 (salida esperada). No necesitas saber si usa Java o C++ para que la prueba sea válida.

2.2. Pruebas de Caja Blanca (White-Box Testing)

Este enfoque es el opuesto. Las pruebas de Caja Blanca (también llamadas de Caja de Cristal o Transparente) implican que **tenemos acceso total y conocimiento completo del interior del sistema**.

El tester (que suele ser el propio desarrollador) puede ver el código fuente, la lógica, la arquitectura y cómo están estructurados los datos.

El objetivo no es probar el comportamiento final, sino probar la **lógica interna** y la **estructura del código** para asegurar que sea robusta, segura y eficiente.

- **Se enfoca en:** "¿El sistema está *construido correctamente* por dentro?"
- **Perspectiva:** La del **Desarrollador**.
- **Se basa en:** El diseño técnico y el código fuente.

- **Analogía:** El mecánico de autos. Abre el capó (accede al interior), conecta una computadora de diagnóstico y mide el voltaje de los sensores y la presión del combustible. No prueba si el auto "llega a 100 km/h" (eso es Caja Negra), prueba si los componentes internos funcionan según su especificación.

2.3. Pruebas de Caja Gris (Grey-Box Testing)

Como su nombre lo indica, es una mezcla de las dos anteriores. El tester de Caja Gris **no conoce el código fuente** (como en Caja Blanca), pero sí tiene **conocimiento parcial** de la arquitectura interna.

El tester sabe, por ejemplo, cómo se llama la tabla en la base de datos, qué arquitectura de microservicios se usa o cómo leer los *logs* (registros) del servidor.

Ejemplo:

1. **Prueba (Caja Negra):** El tester intenta registrar un nuevo usuario ("pepe@mail.com") desde la página web.
2. **Resultado:** La página da un error genérico: "Error al registrar".
3. **Análisis (Caja Gris):** El tester (que tiene acceso a la base de datos) ejecuta una consulta: `SELECT * FROM usuarios WHERE email = 'pepe@mail.com'`.
4. **Diagnóstico:** Descubre que el usuario SÍ se creó en la base de datos. El problema no fue el registro, sino que la API (la lógica del *backend*) le devolvió un mensaje de error incorrecto a la página web (el *frontend*).

3. Profundización: Técnicas de Pruebas de Caja Negra

Probar "al azar" no es eficiente. Si un campo de texto acepta números del 1 al 1.000.000, no podemos probar un millón de veces. La ingeniería de software nos da técnicas para diseñar un conjunto pequeño de pruebas que encuentren la mayor cantidad de errores.

3.1. Técnica 1: Partición de Equivalencia

Esta técnica consiste en dividir *todos* los posibles datos de entrada en "clases" o "particiones" donde se espera que el sistema se comporte de la misma manera. Luego, se prueba *un solo valor* de cada clase.

Ejemplo: Un sistema de inscripción a la universidad acepta edades de **18 a 65 años**.

No necesitamos probar 18, 19, 20, 21... Las particiones de equivalencia serían:

- **P1 (Inválida):** Edad < 18 (Ej. 10)
- **P2 (Válida):** 18 <= Edad <= 65 (Ej. 35)
- **P3 (Inválida):** Edad > 65 (Ej. 70)
- **P4 (Inválida):** No numérico (Ej. "ABC")
- **P5 (Inválida):** Vacío (Ej. "")

Con solo 5 pruebas, hemos cubierto la lógica de millones de posibles entradas.

3.2. Técnica 2: Análisis de Valores Límite (BVA)

Esta técnica complementa a la anterior. Se basa en la experiencia de que la mayoría de los errores de programación no ocurren en el medio de un rango, sino en los **"bordes" o "límites"**.

Los programadores a menudo cometen errores usando > (mayor) en lugar de >= (mayor o igual).

Usando el mismo ejemplo (edades 18 a 65), el BVA nos dice que debemos probar los valores *exactamente* en el límite, y *justo al lado* de él.

- **Límite Inferior (18):**
 - 17 (Inválido)
 - 18 (Válido)
 - 19 (Válido)
- **Límite Superior (65):**
 - 64 (Válido)
 - 65 (Válido)
 - 66 (Inválido)

Combinando Partición de Equivalencia y BVA, tenemos un conjunto de pruebas de Caja Negra muy potente.

3.3. Técnica 3: Tablas de Decisión

Esta técnica es excelente para probar **reglas de negocio complejas** que dependen de múltiples condiciones.

Ejemplo: Un sitio de e-commerce tiene una regla para un descuento: "Si el cliente es 'Premium' Y la compra es 'mayor a \$50.000', aplicar 'Descuento del 15%'".

Aquí tenemos 2 condiciones, lo que genera 4 combinaciones posibles (2^2). Una tabla de decisión nos ayuda a no olvidar ninguna:

Regla	Condición 1: ¿Cliente Premium?	Condición 2: ¿Compra > \$50.000?	Acción: ¿Aplicar Descuento?
1	Verdadero	Verdadero	Sí
2	Verdadero	Falso	No
3	Falso	Verdadero	No
**4. **	Falso	Falso	No

Esta tabla se convierte en 4 pruebas de Caja Negra que garantizan que la lógica de negocio (no solo un campo) funciona correctamente.

4. Profundización: Técnicas de Pruebas de Caja Blanca

4.1. El Objetivo: Cobertura de Código (Code Coverage)

El objetivo principal de la Caja Blanca es lograr la mayor **Cobertura de Código** posible. Esta es una métrica (un porcentaje) que indica qué parte de nuestro código fuente ha sido "tocada" o ejecutada por nuestras pruebas.

Si escribimos 100 líneas de código, y nuestras pruebas solo ejecutan 80 de ellas, tenemos un 80% de cobertura. Esas 20 líneas sin probar son un riesgo: podrían contener un error crítico que nunca vimos.

Existen varios niveles de cobertura:

4.2. Cobertura de Sentencia (Statement Coverage)

Es la métrica más simple. Mide cuántas líneas (o sentencias) de código se ejecutaron.

Pseudocódigo:

1. Funcion calcular_promedio(a, b):
2. total = a + b
3. promedio = total / 2
4. Imprimir(promedio)

Una sola prueba (Ej. calcular_promedio(10, 20)) ejecutaría las 4 líneas, dándonos un **100% de cobertura de sentencia.**

4.3. Cobertura de Decisión (Decision/Branch Coverage)

Esta es mucho más importante. Mide si hemos probado todos los posibles resultados de una decisión (como un IF o un CASE).

Pseudocódigo:

1. Funcion es_mayor_de_edad(edad):
2. IF edad >= 18:
3. Imprimir("Es mayor")
4. ELSE:
5. Imprimir("Es menor")

- **Prueba 1:** es_mayor_de_edad(25)
 - Ejecuta las líneas 1, 2 y 3.
 - **Cobertura de Sentencia:** 3/5 (60%).
 - **Cobertura de Decisión:** 1/2 (50%). Solo probamos el camino TRUE del IF.
- **Prueba 2:** es_mayor_de_edad(10)
 - Ejecuta las líneas 1, 2, 4 y 5.
 - **Cobertura de Sentencia:** 4/5 (80%).

- **Cobertura de Decisión:** 1/2 (50%). Solo probamos el camino FALSE del IF.

Para lograr un **100% de Cobertura de Decisión**, estamos *obligados* a ejecutar **ambas pruebas**.

4.4. Cobertura de Bucle (Loop Coverage)

Mide si hemos probado los bucles (FOR, WHILE) correctamente. Un bucle tiene tres escenarios de riesgo:

1. **Cero iteraciones:** ¿Qué pasa si el bucle nunca se ejecuta? (Ej. procesar una lista vacía).
2. **Una iteración:** ¿Qué pasa si el bucle se ejecuta una sola vez?
3. **N iteraciones:** ¿Qué pasa si el bucle se ejecuta múltiples veces? (El "camino feliz").

Las pruebas de Caja Blanca nos fuerzan a pensar en estos escenarios que las de Caja Negra podrían pasar por alto.

5. Los Niveles de Prueba: ¿Cuándo y Qué Probamos?

Los enfoques (Negro, Blanco, Gris) no son mutuamente excluyentes. Se aplican en diferentes **Niveles** del proceso de desarrollo. Los niveles responden a la pregunta: ¿Estamos probando una sola función, dos módulos juntos, o el sistema completo?

5.1. La Pirámide del Testing (Diagrama)

La "Pirámide del Testing" es un modelo visual que nos dice *cuánto* esfuerzo (y cuántas pruebas) deberíamos dedicar a cada nivel. La regla es: "Escribe muchas pruebas pequeñas y rápidas (base), y pocas pruebas grandes y lentas (punta)".

5.2. Nivel 1 (Base): Pruebas Unitarias (Unit Tests)

- **¿Qué es?** Pruebas que verifican la pieza de código más pequeña posible (una sola función, método o clase) de forma **aislada** del resto del sistema.
- **Enfoque:** 100% **Caja Blanca**. El desarrollador que escribe la función es quien escribe la prueba.
- **Ejemplo:** Probar que la función calcularIVA(100) devuelve 21. Para "aislarla", si la función depende de la AFIP, se simula una respuesta de la AFIP.
- **Pirámide:** Son la base. Deben ser miles. Son **rápidas** (milisegundos) y **baratas** de escribir y ejecutar.

5.3. Nivel 2: Pruebas de Integración (Integration Tests)

- **¿Qué es?** Pruebas que verifican que dos o más "unidades" (módulos, servicios, componentes) funcionan correctamente *juntos*.
- **Enfoque:** Generalmente **Caja Gris**.
- **Ejemplo:** Probar que tu API (backend) puede conectarse, escribir y leer exitosamente en la Base de Datos. O probar que el módulo de "Carrito" puede comunicarse con el módulo de "Stock" para descontar un ítem.
- **Pirámide:** Son el medio. Hay menos que las unitarias. Son más **lentas** (requieren conectar servicios, redes, BDs).

5.3.1. Enfoques de Integración

No es lo mismo integrar todo de golpe que de a poco:

- **Big Bang:** Se conectan *todos* los módulos al final y se reza para que funcione. Es una mala práctica que genera caos.
- **Incremental:** Se van conectando los módulos uno por uno. Es la forma recomendada.
 - **Top-Down (De arriba hacia abajo):** Se prueba desde la Interfaz de Usuario (UI) hacia la base de datos, simulando las capas inferiores que aún no existen (con "Stubs").
 - **Bottom-Up (De abajo hacia arriba):** Se prueba desde la Base de Datos hacia la UI, simulando las capas superiores (con "Drivers").

5.4. Nivel 3: Pruebas de Sistema (System Tests)

- **¿Qué es?** Pruebas que evalúan el sistema *completo* y totalmente integrado, de principio a fin (End-to-End).
- **Enfoque:** 100% **Caja Negra**. Se prueba el *comportamiento* del sistema final, como lo haría un usuario.
- **Ejemplo:** Simular un flujo de usuario completo: "Abrir la web, buscar un producto, agregarlo al carrito, ir a pagar, ingresar datos de tarjeta y recibir la confirmación de compra".
- **Pirámide:** Cerca de la cima. Hay pocas, son **lentas** (requieren abrir un navegador real) y **frágiles** (se rompen si cambia un botón de lugar).

5.4.1. Pruebas Funcionales vs. No Funcionales

Las Pruebas de Sistema se dividen en dos grandes familias:

- **Pruebas Funcionales:** Verifican **el qué**. ¿El sistema *hace* lo que dice que hace? (Ej. ¿El botón de login permite ingresar?).
- **Pruebas No Funcionales:** Verifican **el qué tan bien**. ¿El sistema lo hace *bien*? (Ej. ¿El login es rápido, seguro y fácil de usar?).

5.4.2. Tipos Clave de Pruebas No Funcionales

(Esta es una categoría que la Unidad 8 no cubre y es fundamental)

- **Pruebas de Performance (Rendimiento):** Miden la velocidad, estabilidad y escalabilidad del sistema.
 - **Pruebas de Carga (Load Testing):** Se simula la cantidad *esperada* de usuarios (Ej. "500 usuarios concurrentes en el Hot Sale"). ¿El sistema resiste?
 - **Pruebas de Estrés (Stress Testing):** Se aumenta la carga hasta encontrar el "punto de quiebre". (Ej. "¿A los cuántos usuarios se cae el sistema: 5.000, 10.000?").
- **Pruebas de Usabilidad:** Evalúan qué tan fácil, intuitivo y amigable es el sistema para el usuario. (Ej. ¿El botón de "Ayuda" es fácil de encontrar?).
- **Pruebas de Seguridad (Introducción):** Intentan "hackear" el sistema para encontrar vulnerabilidades. (Ej. Inyección SQL, Cross-Site Scripting (XSS)).

5.5. Nivel 4 (Punta): Pruebas de Aceptación (UAT)

- **¿Qué es?** Pruebas que confirman que el software cumple con los requisitos del *negocio* y está listo para que el cliente lo "acepte" formalmente.
- **Enfoque:** 100% **Caja Negra**.
- **Ejemplo:** El dueño de la librería (el cliente) entra al sistema y prueba si el flujo de carga de un nuevo libro es cómodo y funcional *para él*.
- **Pirámide:** La punta. Suelen ser pruebas manuales y finales.

5.5.1. Pruebas Alpha y Beta

- **Pruebas Alpha:** Se realizan *dentro* de la empresa desarrolladora, pero por personas que no son del equipo de desarrollo (Ej. empleados de marketing, soporte, etc.) para tener una visión fresca.
- **Pruebas Beta:** Se libera el producto a un grupo selecto de *usuarios reales externos* (los "Beta Testers") para que lo usen en el "mundo real" antes del lanzamiento oficial.

6. Conclusión: El Puente a la Unidad 8

En esta unidad, hemos sentado las bases teóricas del Testing. Hemos construido nuestro "mapa" para entender esta disciplina.

Aprendimos que no existe una sola forma de probar, sino un conjunto de:

- **Enfoques:** Que responden al *desde dónde* probamos (Caja Negra, Blanca, Gris).
- **Niveles:** Que responden al *cuándo* y *qué* probamos (Unitario, Integración, Sistema, Aceptación).
- **Técnicas:** Que nos ayudan a diseñar pruebas *inteligentes* (Particiones, Límites, Cobertura).

Ahora que ya entiendes *qué* es probar y *por qué* se hace, estás listo para la siguiente etapa, que es la **ejecución y gestión** de este proceso.

En la **Unidad 8**, tu compañero te enseñará el proceso práctico que ocurre *después* de que decidimos qué probar:

- ¿Cómo se **documenta** formalmente una prueba (el "Caso de Prueba")?
- ¿Cómo se **analizan** y **reportan** los resultados (Tipos de Error, Severidad Crítica/Alta)?
- ¿Cómo se **gestiona** el ciclo de corrección de un error (Re-pruebas, Pruebas de Regresión)?
- ¿Cuál es la diferencia final entre **Testing** (detectar) y **Aseguramiento de la Calidad (QA)** (prevenir)?